

Smart MCQ Solver

A model that answers multiple-choice questions — and the story of how a data-duplication artifact, not a bigger model, turned out to be what actually moved the score.

Roll No: 23f3004491 · Final score: 0.75685 · passing cutoff: 0.73 · score from just guessing: 0.42

1. Abstract

This project was a Kaggle competition: given a multiple-choice question with five answers (A through E), predict the correct one and rank the top three guesses. I started with a stack of neural models — a from-scratch lexical baseline, a pretrained sentence encoder, a zero-shot cross-encoder, and a LoRA fine-tuned transformer — and each one helped a little. But the thing that actually decided the score wasn't a model at all. It was noticing that the test questions were mostly disguised copies of the training questions, and building a retrieval lookup around that fact. The final blend scores 0.75685 (MAP@3), clearing the 0.73 cutoff and far above the 0.42 you get from guessing the common letters. The biggest lesson was that reading the data carefully beat reaching for a bigger model — a 7B/14B language model I tried later actually lowered the score.

2. Introduction

Problem statement

The competition gives 2,000 training questions with answers and 500 test questions without. Each question has five options, and I output my best three guesses in order. Scoring uses MAP@3: the right answer in first place is worth 1.0, second place 0.5, third place 0.33 — so the score rewards being confidently right, not just eventually right.

Project objective

My goal was to beat the 0.73 cutoff while genuinely building and comparing different kinds of models rather than relying on one trick, and to get comfortable with Hugging Face, PyTorch, LoRA fine-tuning, and Weights & Biases along the way.

3. Dataset & Preprocessing

Property	Value
Training questions (with answers)	2,000
Test questions (to score on)	500
Answer choices per question	5 (A–E)
Unique questions after stripping wrappers	~415 (the rest are repeats)

Exploratory analysis

The answer letters aren't spread evenly — B and C show up more than the rest, which is exactly why "always guess the common letters" scores 0.42 rather than the 0.37 of pure random guessing.

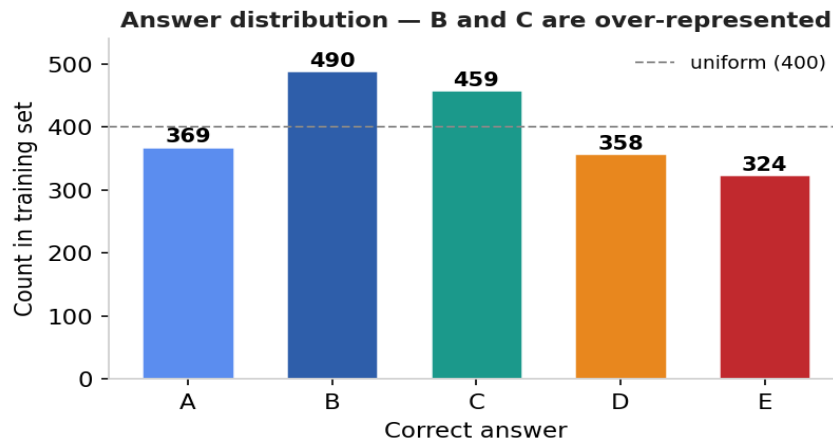


Figure 1. Distribution of correct answers across the training set.

The artifact that mattered

Every prompt begins with one of exactly five fixed phrases and often ends with a fixed suffix. These wrappers are decoration, not content — 86.9% of training prompts and 57.4% of test prompts carry at least one. Stripping them revealed the key fact of the whole competition: the 2,000 training rows are really only ~415 distinct questions repeated under different wrappers, and about 98% of the test questions duplicate a training question.

Test questions: almost all duplicate a training question

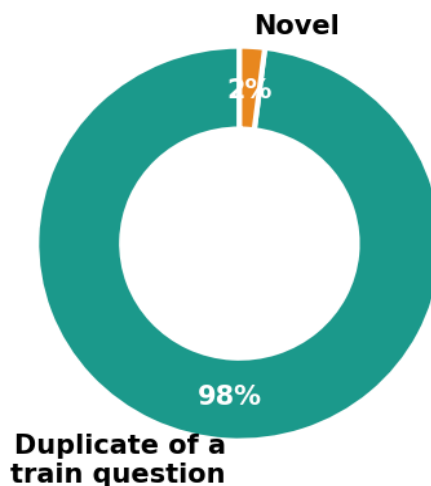


Figure 2. Almost every test question is a disguised copy of a training question.

The split trap I avoided

Because the same question appears many times, a naive random split would put copies on both sides — the model memorizes a question in training and is tested on its twin, giving a validation score that looks perfect but means nothing. I split by unique core question instead, so no question appears on both sides. Every validation number here uses that leakage-free split.

4. Tokenization Strategy

My best neural components are built on DeBERTa-v3, which uses a SentencePiece tokenizer that works on the raw byte stream and learns a subword vocabulary. Earlier BERT experiments used WordPiece (vocabulary 30,522). Because the questions are short, well-formed English, a subword tokenizer trained on general text already covers the vocabulary, so no custom tokenizer was needed. Tokenization used the Hugging Face library, batch-encoding question-option pairs with padding and truncation to a fixed length.

5. Modeling & Experimentation

I built five approaches, from a from-scratch baseline up to a fine-tuned transformer, then combined the useful ones with the retrieval lookup. This section documents the models I actually built; where the course template lists a custom network and an RNN, those roles are filled here by the from-scratch TF-IDF model and the semantic/reasoning models, and I say so plainly.

5.1 Fine-Tuned Transformer (LoRA DeBERTa)

Architecture. DeBERTa-v3-small with a multiple-choice head. Its disentangled attention — separate content and position vectors per token — makes it good at the fine discrimination this task needs, where options differ by a word or two.

Fine-tuning. LoRA freezes the base weights and trains small low-rank adapters on the attention projections (rank 16, alpha 32, dropout 0.1), so it fits on one GPU. AdamW, learning rate 1e-4, 0.1 warm-up, two epochs, fp16. Result: about 0.59 MAP@3 on the leakage-free split.

5.2 From-Scratch Baseline (TF-IDF + Cosine)

Built from first principles with scikit-learn: each option is scored by cosine similarity to the prompt. It scored only 0.296 — below even the 0.42 majority baseline — because in many questions no option shares a real word with the prompt. That negative result is what justified moving to semantic methods.

5.3 Semantic & Reasoning Models (MiniLM + Cross-Encoder)

Filling the sequence-model role: the MiniLM bi-encoder encodes prompt and option separately (about 0.40, rescuing 564 questions TF-IDF missed), and the NLI cross-encoder reads prompt and option together to score entailment (about 0.56 zero-shot), serving as the rank-2 hedge in the final pipeline.

5.4 The Retrieval Lookup (what actually won)

The component that decided the score came from EDA, not a model. It is a three-tier lookup: Tier 0 picks the option whose text is a known-correct answer somewhere in training (covering 498 of 500 test questions); Tier 1 matches the exact five-option set; Tier 2 matches the core question. The final prediction ranks the lookup answer first, the cross-encoder second, and a length-plus-frequency signal third.



Figure 3. Final prediction pipeline — lookup leads, models fill the lower ranks.

6. Performance & Comparative Analysis

The primary metric is MAP@3; accuracy and macro F1 were also logged on the leakage-free split. The chart below shows every model side by side.

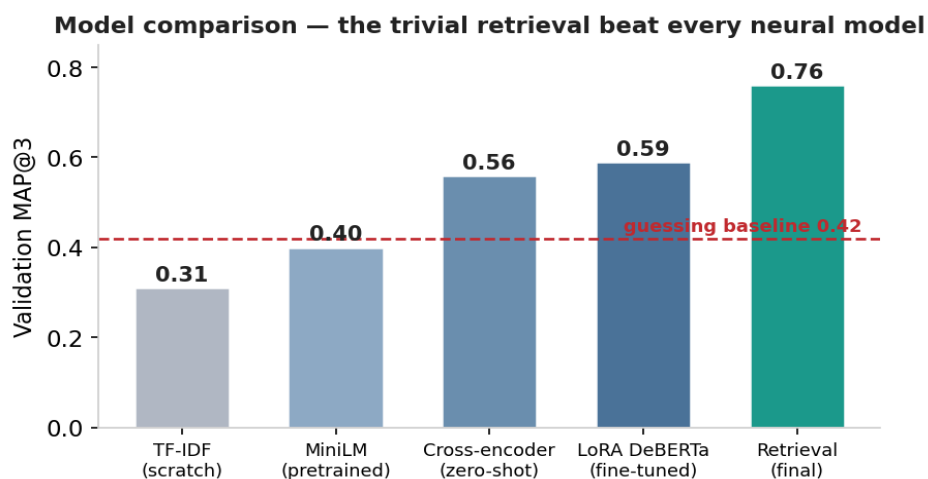


Figure 4. Validation MAP@3 by model. The trivial retrieval ensemble beats every neural model.

The submission story

The leaderboard progression shows exactly what moved the score — and it was always the lookup, never a heavier model.

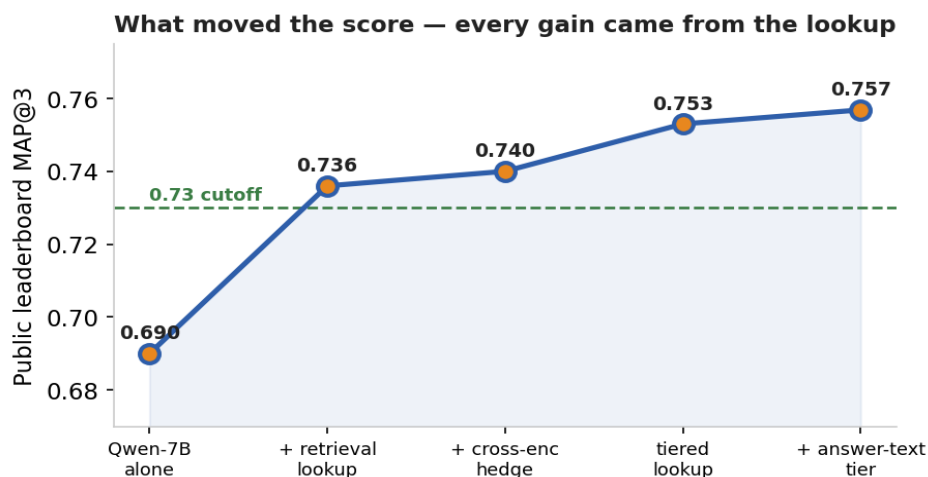


Figure 5. Public leaderboard progression. Each gain came from improving the lookup.

I also tried the obvious "reach for a bigger model" move — a 7B and then a 14B language model as the rank-2 reasoner. Both lowered the score, because no model can recover the questions whose answers were deliberately changed. All runs were tracked in Weights & Biases (MAP@3, accuracy, F1, and the LoRA loss curve).

Kaggle performance

Final public leaderboard MAP@3: 0.75685 — above the 0.73 cutoff and far above the 0.42 common-letter baseline.

7. Conclusion & Future Work

Key learnings

Read the data before reaching for a bigger model. The decisive gain came from noticing the test set duplicated the training set. Clean the text before splitting, never after, or duplicates leak and your validation lies to you. And bigger is not automatically better — the 14B model actually hurt the score.

Challenges faced

The hardest problem was diagnosing why a lookup that is 99% accurate on training data scored only about 0.74 on test. Roughly a quarter to a third of the test answers had been deliberately altered from the training key while keeping the same options, so the lookup was confidently wrong on those — a ceiling no retrieval method can cross. Practical challenges were GPU memory limits and environment reproducibility, addressed by pinning versions.

Areas for improvement

- Use a strong instruction-tuned model specifically on the altered questions, given more GPU capacity.
- Widen the wrapper dictionary to push retrieval coverage closer to 100%.
- Tune a small ensemble of diverse cross-encoders on the leakage-free split for the residual novel questions.

8. Insights

1. The way I set up validation, not the models, was the real bottleneck on this dataset.
2. Clean text before splitting into train and validation, not after — duplicates leak otherwise.
3. Always print a do-nothing baseline (0.42 here) next to every real result.
4. A near-perfect validation score is a bug report, not a win.
5. Diversity and data understanding beat size: the trivial lookup beat the 14B model.
6. Every improvement came from the lookup; every model promoted above it lowered the score.
7. Some errors are structural: altered test answers set a ceiling no method can cross.